

Bot de Registro de Horas para Cuidadoras — Versión AI Agent

n8n + Google Gemini + Notion + Telegram

Marcela Valverde Estrada

Resumen

Este proyecto reconstruye, como ejercicio de aprendizaje, un bot previamente funcional (registro de horas de cuidadoras vía Telegram, con lógica manual de nodos Code/If) usando en cambio un nodo AI Agent central que orquesta múltiples herramientas (tools) sobre una base de Notion. El agente decide de forma autónoma cuándo buscar un turno abierto, cuándo crear un ingreso, cuándo cerrar una salida, cuándo registrar un comentario sobre el turno y cuándo consultar el total de horas e importe a pagar del mes, manteniendo memoria de la conversación por chat de Telegram.

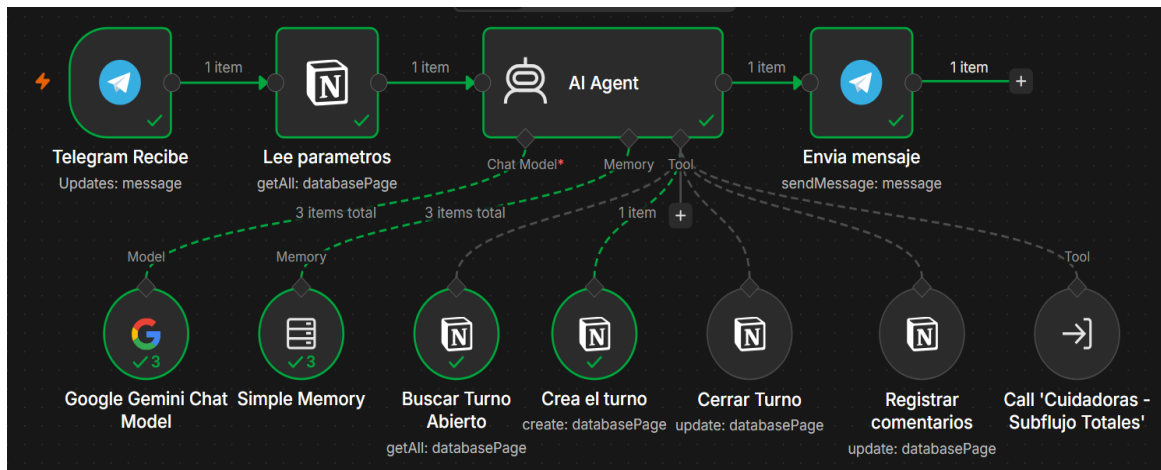
Objetivo

Comparar en la práctica dos formas de resolver el mismo problema de negocio en n8n: una versión manual, con nodos Code e If armando explícitamente cada bifurcación lógica, y una versión con AI Agent + Tools, donde el modelo decide el camino a seguir según la intención del mensaje. El objetivo secundario fue evaluar la confiabilidad de delegar en la IA tareas que involucran cálculo de dinero, y decidir en qué casos conviene mantener ese cálculo en código determinístico en lugar de en el razonamiento del modelo.

Arquitectura del flujo

El workflow principal está compuesto por un nodo AI Agent central, conectado a un modelo de lenguaje, un módulo de memoria y cinco herramientas (tools) que el agente puede invocar según la intención detectada en el mensaje del usuario:

- Telegram Trigger: punto de entrada de la conversación (mensajes de texto libre en español).
- Lee Parámetros: nodo Notion que consulta, antes de invocar al agente, el valor por hora vigente desde una base de "Configuración" separada, evitando que ese valor quede fijo (hardcodeado) en el workflow.
- Google Gemini Chat Model: modelo de lenguaje que interpreta la intención del usuario y decide qué herramienta utilizar.
- Simple Memory: mantiene el historial de la conversación por chat de Telegram (Session ID = chat.id), permitiendo referencias a mensajes anteriores.
- Buscar Turno Abierto (Notion — Get many database pages): busca si una cuidadora ya tiene un turno sin cerrar.
- Crea el turno (Notion — Create database page): registra un nuevo ingreso, usando el valor por hora leído dinámicamente.
- Cerrar Turno (Notion — Update database page): completa la fecha de salida de un turno ya abierto.
- Registrar Comentario (Notion — Update database page): guarda una observación sobre el turno (por ejemplo, una novedad de cuidado).
- Call 'Cuidadoras - Subflujo Totales' (Execute Workflow Tool): invoca un sub-workflow separado que calcula, con código determinístico, el total de horas e importe a pagar del mes, filtrando opcionalmente por una o más cuidadoras.



Arquitectura completa del agente: un modelo central con cinco herramientas conectadas, más memoria y lectura previa de configuración.

Fuente de datos: base de Notion

Los turnos se gestionan en una base de Notion con las columnas Fecha Hora Ingreso, Fecha Hora Salida, Cantidad de Hs. (fórmula), Asistente, Valor por hora, Importe a pagar (fórmula) y Pagado. El agente solo escribe los campos de texto/fecha; el cálculo de horas e importe queda resuelto por las fórmulas nativas de Notion, no por el agente ni por código.

Fecha Hora Ingreso	Fecha Hora Salida	Cantidad de Hs.	Asistente	Valor por hora	Importe a pagar	Pagado
September 4, 2026 7:30	September 4, 2026 13:00	5.5	Mónica			
September 4, 2026 13:00	September 4, 2026 22:00	9	Mónica			
September 4, 2026 22:00	September 5, 2026 9:00	11				
September 5, 2026 9:00	September 5, 2026 13:00	4	Mónica			
September 5, 2026 15:00	September 6, 2026 13:00	22				
September 6, 2026 16:30	September 6, 2026 22:00	5.5				
September 6, 2026 22:00	September 7, 2026 7:30	9.5	Sofía			
September 7, 2026 7:30	September 7, 2026 13:00	5.5	Gigi			
September 7, 2026 13:00	September 7, 2026 22:00	9	Mónica			

Base "Cuidadoras" en Notion, con las fórmulas de horas e importe ya resueltas por Notion.

Problemas técnicos resueltos durante el desarrollo

1. Claves de \$fromAI() duplicadas con descripciones distintas

Al usar la misma clave (por ejemplo "cuidadora" o "page_id") en más de una herramienta con una descripción de texto levemente distinta en cada una, n8n rechazó la ejecución con el error "Duplicate key found with different description or type". Se resolvió unificando el texto exacto de la descripción, copiándolo literalmente entre las herramientas que comparten la misma clave, en lugar de reescribirlo cada vez.

2. Columnas tipo Select no permiten valores dinámicos desde la interfaz

Los campos de tipo Select de Notion (como "Asistente") no ofrecen el botón "From AI" en la interfaz, ni aceptan una expresión libre en modo Fixed — solo permiten elegir de una lista fija de opciones ya creadas. Se resolvió escribiendo manualmente la función `{{ $fromAI('clave', 'descripción', 'string') }}` en modo Expression, que sí es aceptada aunque el botón visual no esté disponible para ese tipo de columna.

3. Imposibilidad de agregar propiedades cuando el ID de página es una expresión

En los nodos "Update a database page" usados como Tool, con el campo Page en modo Expression, el selector de propiedades fallaba con "Error fetching options from Notion Tool", porque n8n no puede consultar el esquema de una base sin resolver primero a qué página pertenece. Se resolvió con un cambio temporal: fijar el campo Page en modo Fixed con un ID real existente, agregar la propiedad desde la lista (que entonces sí se completa con el tipo correcto), y recién después volver a poner Page en modo Expression con la función `$fromAI` correspondiente.

4. Prompt del agente ubicado en el campo incorrecto

Las instrucciones generales del agente (System Prompt) se pegaron por error en el campo "Prompt (User Message)", que en realidad debe contener el mensaje real del usuario (`{{ $json.message.text }}`). Se resolvió moviendo las instrucciones al campo "System Message", dentro de "Options", y dejando el campo de mensaje de usuario con la referencia al texto real de Telegram.

5. Referencia rota al insertar un nodo intermedio

Al insertar el nodo "Lee Parámetros" entre el trigger de Telegram y el AI Agent, la expresión `{{ $json.message.text }}` dejó de funcionar, porque `$json` pasó a referirse a los datos de Notion (el nodo inmediatamente anterior), no al mensaje de Telegram. Se resolvió reemplazando la referencia por `{{ $('Telegram Trigger').first().json.message.text }}`, que apunta al nodo por nombre en lugar de depender de su posición en el flujo.

6. Credencial de IA incompatible con el nodo

El Chat Model del AI Agent falló con "Gateway credits eligibility check failed: nodeNotCovered", porque venía preconfigurado con el sistema de créditos propio de n8n, que no cubre este nodo específico. Se resolvió creando una credencial propia de tipo "Google Gemini(PaLM) API account", con la misma API key generada en Google AI Studio.

Comparación con el enfoque manual (Code + If)

La versión previa del mismo bot, armada con nodos Code e If explícitos, resultó más predecible y depurable paso a paso: cada bifurcación y cada dato intermedio podía inspeccionarse directamente en el JSON de cada nodo. La versión con AI Agent necesitó menos nodos de lógica manual (el propio agente decide el camino), pero es más difícil de depurar cuando falla, consume más llamadas al modelo por mensaje (y por lo tanto más tiempo y costo), y en al menos un caso normalizó un nombre propio de forma inesperada ("Mónica" interpretado como "Moni"), lo que obligó a agregar al System Prompt una lista cerrada de nombres válidos.

Consideración de seguridad

Las herramientas de creación y actualización de registros (Crear Ingreso, Cerrar Turno, Registrar Comentario) fueron implementadas para uso personal, con fines de aprendizaje. En un entorno con más de un usuario, no sería recomendable dejar que el agente escriba directamente sobre la base sin ningún paso de confirmación intermedio, ya que un mensaje ambiguo o mal interpretado podría generar o modificar un registro incorrecto sin que nadie lo revise antes.

Por decisión de diseño, el cálculo de totales de horas e importe a pagar (que involucra dinero) se delegó deliberadamente a un sub-workflow con código determinístico, y no al razonamiento del agente sobre los números — un LLM sumando muchos valores a mano no es una operación confiable para este tipo de dato.

Stack tecnológico

- n8n — orquestación del agente, las herramientas y el sub-workflow de cálculo.
- Google Gemini (Flash) — modelo de lenguaje utilizado por el AI Agent.
- Notion API — almacenamiento de turnos, comentarios y configuración (valor por hora vigente).

- Telegram Bot API — canal de entrada y salida de la conversación.

Resultado

Agente conversacional funcional, capaz de decidir de forma autónoma entre cuatro operaciones distintas (ingreso, salida, comentario y consulta de totales) dentro de la misma conversación, manteniendo contexto por sesión de Telegram, con el cálculo monetario resuelto por código determinístico invocado como herramienta en lugar de por el modelo. Quedó documentada la resolución de seis problemas técnicos reales, y una comparación práctica entre el enfoque manual (Code + If) y el enfoque con AI Agent + Tools para este mismo caso de uso.